USER GUIDE

Coral

Sprite atlas & 2D animation toolkit

Coral is a desktop toolkit for 2D sprites and animation: **pack sprites into an atlas** and export it for Godot, Unity, Spine and Phaser / PixiJS (Pack), **cut an existing sprite sheet** (Slice), **play animations, set pivots and draw hitboxes** (Animate), **lay out VFX frame sequences as a SubUV sheet** (Flipbook), **pixel-art tools** — upscale, palettes, colour swaps (Pixel) — and **make normal maps** for 2D lighting (Normal 2D).

Version: 0.1.0 · **Generated:** 2026-11-09

The same content as the in-app Help (press F1).

Contents

1 **Getting started**
What Coral is, the layout, drag & drop, projects, how Coral writes files safely, shortcuts, the log and appearance.

2 **Pack — atlas packing**
Turn PNG / TGA, Aseprite, PSD and GIF sources into an atlas (trim, duplicate merging, MaxRects) and export JSON (Phaser, PixiJS, Paper2D), Godot, Unity and Spine. Watch re-exports on save.

3 **Slice — cut a sprite sheet**
Cut an existing sheet by grid or automatically, edit rects, names, pivots and 9-slice; write separate PNGs or metadata (JSON Hash, Unity .meta) for the uncut sheet.

4 **Animate — playback, pivots, hitboxes**
Play the animations of the project open in Pack, edit FPS / loop mode / per-frame durations, onion skin, set pivots and draw hitboxes / hurtboxes per frame.

5 **Flipbook — SubUV sheets for VFX**
Lay out a numbered frame sequence as an even N×M sheet (SubUV) for Niagara / Unity particles, with an optional JSON.

6 **Pixel — upscale, palettes, swaps, bleed**
Four batch tools for pixel art: Upscale (nearest neighbour), Remap to a palette (OKLab), Swap to make colour variants, and Bleed for transparent pixels.

7 **Normal 2D — normal maps for 2D lighting**
Make normal maps from luminance or an alpha bevel, OpenGL / DirectX convention, with a lit preview that follows the pointer; reuse the settings for Pack's normal atlas.

1 Getting started

What Coral is, the layout, drag & drop, projects, how Coral writes files safely, shortcuts, the log and appearance.

What is Coral?

Coral is a desktop toolkit for 2D sprites and animation: **pack sprites into an atlas** and export it for Godot, Unity, Spine and Phaser / PixiJS (Pack), **cut an existing sprite sheet** (Slice), **play animations, set pivots and draw hitboxes** (Animate), **lay out VFX frame sequences as a SubUV sheet** (Flipbook), **pixel-art tools** — upscale, palettes, colour swaps (Pixel) — and **make normal maps** for 2D lighting (Normal 2D).

- The app's interface is in English; this guide shows button and option names in **bold** exactly as on screen.
- Open this guide any time with `F1` (it opens on the module you are in), the **?** button in the header, or Ctrl+K → **Help / Hướng dẫn**.

Layout

- **Navigation rail (left)**: the 6 modules, in the order of Ctrl+1 … Ctrl+6 — Pack, Slice, Animate, Flipbook, Pixel, Normal 2D. At the bottom: **Settings** (Ctrl+,), **About & contact**, **Command palette** (Ctrl+K) and **Show log / Hide log** (with an error count while the log is closed).
- **Header**: `Coral / <module>`, the project name (`(unsaved)` before the first save, `•` when there are unsaved changes), the **Search actions…** box (opens the command palette), the **?** button (this guide) and the version pill (`browser preview` outside the desktop app; `via Abyss` when Abyss launched it).
- **Log (bottom)**: messages from the UI and the Rust backend. Toggle it with Ctrl+`; Ctrl+K → **Clear log** empties it.

Opening files and folders

- **Drag & drop** files or folders anywhere on the window: they load into **the module you are in**. Each module takes different inputs (see its topic).
- Pack and Animate: drop folders / `.aseprite` / `.psd` / `.gif` to add sources; drop a `*.coral.json` file to open that project.
- Slice: one PNG / TGA / GIF sheet. Flipbook: a folder of frames. Pixel and Normal 2D: images or folders (added to the input list).
- Ctrl+K → **Add a folder…** picks a folder and sends it to the current module, as if you had dropped it.

Projects (*.coral.json)

A project holds what Pack and Animate use: sprite sources, pack settings, export targets, edited pivots, edited animations, hitboxes, and colour-swap tables (when you click **Save to project** in Pixel). Paths inside it are **relative** to the project file, so it can live in the game repo.

- **Open…** / **Save** in Pack's top bar, or Ctrl+O / Ctrl+S / Ctrl+Shift+S (save as). The first save asks where, defaulting to `<atlas>.coral.json`.
- Ctrl+K → **New project** starts an empty project (stops Watch; it does not ask to save — save first). **Recent projects** in the command palette reopens the last 10.
- Slice, Pixel and Normal 2D settings are **not** stored in the project. Flipbook's options (except the frame list) are remembered by the app between sessions.

How Coral writes files

- **Pack** and **Slice** always list the files first (the **Write N file(s)?** dialog, each line tagged `will write`); nothing is written until you click **Write files**.
- **Flipbook**, **Pixel** and **Normal 2D** write as soon as you click the main button. Pixel and Normal 2D have a **Dry run** button to see the list first; Flipbook re-plans by itself whenever an option changes.
- Every file is written **atomically**: to `<file>.tmp`, then renamed. An existing file at the same path is **replaced**.
- **In place (.bak)** (Pixel) copies each input to `<file>.bak` before overwriting it. An existing `.bak` is kept as is (the first original survives).
- Coral never deletes your files. Click a written file's path in a result list to show it in Explorer.

Shortcuts

`F1`	Open / close this guide (on the current module's topic)
`Ctrl` `K`	Command palette: modules, project actions, recent projects, updates…
`Ctrl` `1` … `Ctrl` `6`	Switch module (Pack, Slice, Animate, Flipbook, Pixel, Normal 2D)
`Ctrl` `O`	Open a project
`Ctrl` `S` / `Ctrl` `Shift` `S`	Save the project / save it as
`F5`	Run the current module again (never writes): Pack / Animate re-pack the preview, Slice runs **Detect** again, Flipbook re-plans, Normal 2D refreshes its preview; nothing in Pixel
`Ctrl` `` ` ``	Show / hide the log
`Ctrl` `,`	Settings (appearance)
`Esc`	Close a dialog / the command palette / this guide

- In Slice, `F5` runs **Detect** again and **replaces every rect** — hand edits (positions, names, pivots, 9-slice) are lost.

Appearance and updates

- **Settings** (Ctrl+,): light / dark / Windows theme, UI and code fonts, UI scale, effects level.
- Ctrl+K → **Check for updates…** (or in **About & contact**). When a new version exists, the header shows **Update to vX.Y.Z** to download, install and restart.
- When Abyss launches Coral, Coral does not check by itself: Abyss shows the update and installs it when the app closes.

2 Pack — atlas packing

Turn PNG / TGA, Aseprite, PSD and GIF sources into an atlas (trim, duplicate merging, MaxRects) and export JSON (Phaser, PixiJS, Paper2D), Godot, Unity and Spine. Watch re-exports on save.

How to use

1. **Add sources**: drop a folder of PNG / TGA files, `.aseprite` / `.ase`, `.psd` or `.gif` files onto the window, or use the folder button (**Add a folder**) of the **Sources** card. A loose PNG adds **its folder** (without sub-folders, that file type only).
2. Set the **atlas name** in the box at the left of the top bar (it names the exported files). An unsaved project takes the first source's name.
3. Watch the **atlas** in the middle: it re-packs after every edit. Adjust the **Pack** settings on the right.
4. In the **Exports** card, click +, choose a target (**JSON Hash**, **JSON Array**, **Godot**, **Unity**, **Spine**) and pick its output folder.
5. Click **Pack & Export**: Coral lists the files it will write; click **Write files** to write them.
6. **Save** (Ctrl+S) the project, then turn on **Watch** to re-export whenever you save a source file.

Sources

Click a source to edit it; × removes it from the project (no file is deleted).

- **Folder**: **Include subfolders** (on for a dropped folder) and a comma-separated list of include patterns (default `*.png, *.tga`).
- **Aseprite**: **Visible merged** (visible layers merged), **All merged** (every layer merged) or **Per layer** — type the layer names to split, comma separated (e.g. `Body, FX`); each layer becomes its own set of sprites. Tags become animations; frame durations, slice pivots and 9-slice are kept.
- **PSD**: **One sprite per layer** (default) or **Flattened** (the whole file as one sprite). Only pixel layers are read (no effects / smart objects).
- **GIF**: each frame becomes a sprite `<file>_<i>`.
- Sprite names: the path inside the folder without the extension, sub-folders joined with `/` (e.g. `fx/spark_0`); Aseprite: `<file>_<tag>_<i>`. Two sprites with the same name are a `duplicate-name` error — Coral never renames silently.
- Animations come from Aseprite tags and numbered names (`run_0, run_1…`, `walk0001…`). Play and edit them in the **Animate** module.

Atlas preview

- Zoom 25 %–800 % with the buttons above. Hover a sprite to see its name; click it to select it — the bar below shows its frame in the atlas, source size, trimmed area and pivot.
- When one page is not enough, pages appear as **Page 1**, **Page 2**… With **Normal-map atlas** on, the **Normal map** box shows the normal atlas.
- The **Sprites** list on the left has a **Filter…** box. `alias` marks a sprite with the same pixels as another one: they share one rect in the atlas.

- The footer: sprites, pages, aliases, fill %, animations and how long the preview took to pack.

Pack settings

- **Max size** (512–8192, default 2048): the largest page side. **Image**: **PNG** or **TGA**.
- **Padding** (default 2 px): space between sprites. **Extrude** (default 1 px): repeats the edge pixels outward so texture filtering does not pull in neighbours.
- **Trim** cuts away transparent borders; **Trim alpha ≤** treats pixels at or below that alpha as empty. The original size and offset are exported, so sprites stay in place in the engine.
- **Merge duplicates**: identical sprites share one rect (alias). **Power of two** / **Square**: page sizes are powers of two / square. **Multiple pages**: add pages when one is not enough (off: sprites that do not fit are an error).
- **Allow rotation**: rotates sprites 90° for a tighter fit — **not supported by Godot or Unity**: exports for them are blocked.
- **Alpha bleed**: fills transparent pixels with neighbouring colours (alpha stays 0) so filtering / mipmaps show no dark fringes. **Premultiply**: premultiplies colour by alpha (only if your engine expects it).
- **Normal-map atlas**: a second atlas `<name>_n` with the same rects; its settings come from the **Normal 2D** module (**Use for the project's normal atlas**).
- **Placement**: how MaxRects picks a spot — **Best short side** (default), **Best long side**, **Best area**, **Bottom left**, **Contact point**. Try a few for a smaller atlas.

Export targets and their files

Each target writes **its own copy of the atlas image(s)** into its folder (`<name>.png`, or `<name>-0.png`, `<name>-1.png`… with several pages; `<name>_n.png` for the normal atlas) plus the data file below. `<name>` is the atlas name.

- **JSON Hash** (TexturePacker format — Phaser 3, PixiJS): `<name>.json`. Tick **Unreal Paper2D (.paper2dsprites)** to give the data file the `.paper2dsprites` extension for Paper2D. **JSON Array**: same content, frames as an array. FPS, loop modes, frame durations, hitboxes and 9-slice are in `meta.coral`.
- **Godot** (Godot 4): `<name>.tres` (one SpriteFrames with every animation; ping-pong is unrolled into frames). The output folder must be inside a Godot project — `project.godot` is searched upward, or click the **Godot project:** line to set it. **Also one AtlasTexture .tres per sprite** for UI sprites.
- **Unity** (Unity 6): `<name>.png.meta` (Sprite Mode Multiple) with **Pixels per unit** (default 100) and **Filter Point** / **Bilinear**. Exporting again **keeps the meta's guid**, so references in scenes survive.
- **Spine** (Spine / libGDX): `<name>.atlas`, filter **Linear** or **Nearest**.
- Click a target's path to change its folder; the bin removes the target. A target without a folder shows a red **Choose a folder…** and blocks the export.

Pack & Export

1. Click **Pack & Export** (at least one export target is needed).
2. The **Write N file(s)?** dialog lists every file (`will write`). Files at those paths are replaced; an existing Unity `.meta` keeps its guid.
3. Click **Write files**. The **Last export** card shows the result and time; click a path to show it in Explorer.

- **Export blocked**: there are errors (an unreadable source, duplicate sprite names, a sprite larger than the atlas, rotation with a Godot / Unity target, a missing folder…). Nothing is written; the **Issues** card says what and where.

Watch

- **Watch** is available once the project has sources and at least one export target. It packs and exports again whenever a source file (or the saved project file) changes — save in Aseprite and the engine hot-reloads.
- Watch **writes without asking** on every run, into the export folders you set.
- The status pill shows the last run; it turns red on an error — click it for the log. **Stop** ends it. Watches stop when the app closes; `"watch": { "autoStart": true }` in a project starts one when it opens.

Command line

The same exe runs headless when its first argument is a subcommand, e.g. `coral.exe pack hero.coral.json` (every export of the project); add `--target godot --dry-run` to list the files only, `--watch` to keep watching, `--report pack.json` for a report. See `coral.exe --help` and Coral's README.

3 Slice — cut a sprite sheet

Cut an existing sheet by grid or automatically, edit rects, names, pivots and 9-slice; write separate PNGs or metadata (JSON Hash, Unity .meta) for the uncut sheet.

How to use

1. Drop a PNG / TGA / GIF sheet onto the window, or click **Choose a sheet...**. Coral runs **Detect** right away (**Automatic** mode).
2. Choose the **Mode**: **Automatic** or **Grid**, adjust the numbers, then click **Detect**.
3. Edit the rects on the sheet (see below); set names, pivots and 9-slice in the **Rect N** card.
4. In the **Write** card, choose the output, then click **Slice N sprite(s)...**: Coral lists the files; click **Write files** to write them.

Automatic and Grid

- **Automatic**: finds every separate shape (pixels with alpha above **Alpha** >, 8-connected). Shapes within **Merge within** px (default 2) become one sprite; specks under **Ignore under** pixels (default 4) are skipped. Rects come in reading order: rows top to bottom, left to right.
- **Grid**: cuts fixed **Cell W** × **Cell H** cells (default 32 × 32), starting at **Offset X / Y**, with **Gap X / Y** between cells. Empty cells are dropped unless **Keep empty cells** is on.
- **Detect** (and `F5`) replaces **every** rect — hand edits are lost. Settle the mode and numbers first, then edit by hand.

Editing rects

- Drag a rect to move it; drag the selected rect's bottom-right corner to resize it. Index numbers show on the rects from 200 % zoom.
- **+ Rect**, then drag on the sheet to add a rect. The bin on the **Rect N** card deletes the selected rect.
- The **Rect N** card: name, exact **X / Y / W / H**, **Pivot** (9 presets or X / Y values 0–1; **Use this pivot for every rect** applies it to all) and **9-slice borders** (**L / T / R / B**, shown as green dashed lines).

Names

The **Names** card: a pattern such as `{base}_{i}` — `{base}` = the sheet's name, `{i}` = index from 0, `{i:03}` = zero-padded (`007`), `{i1}` = counting from 1. **Rename all** applies it to every rect (Detect uses it too).

Output

- **Separate PNGs**: one PNG per rect in the **Output folder** (default `<sheet>_sprites` next to the sheet).
- **JSON Hash**: `<sheet>.json` describing the rects of the uncut sheet (no image is cut), written to the **Output folder**.
- **Unity .meta**: `<sheet>.png.meta` **next to the sheet** — Unity slices it on import.
- **Cannot slice**: two rects share a name or a rect lies outside the sheet. Fix it and try again.

- Slice's rects, names and pivots are not saved in the project and are lost when you open another sheet — write them before switching.

4 Animate — playback, pivots, hitboxes

Play the animations of the project open in Pack, edit FPS / loop mode / per-frame durations, onion skin, set pivots and draw hitboxes / hurtboxes per frame.

Where animations come from

Animate works on the project open in **Pack** (its atlas preview). Animations come from Aseprite tags and numbered names (`run_0…`, `walk0001…`). Add sources in Pack first — dropping sources on Animate adds them to the project too.

> - Editing an animation stores it in the project (the **edited** badge in the list). **Save** the project (Ctrl+S) to keep the changes.

Player

> - **Play** / **Pause**, ◀ ▶ to step one frame. Pick an animation in the **Animations** list on the left (frames · FPS).
> - The **Playback** card: **FPS**, **Loop** (**Loop**, **Once**, **Ping-pong**) and **Onion skin** (0–3 frames before and after, fading from 30 %).
> - The strip under the frame: click to jump to a frame; the number under each frame is its duration (ms) — type a value to change that frame. **Same duration for every frame** drops the per-frame values and goes back to FPS.
> - A red dot under a frame means it has boxes.

Pivot

1. Pick the crosshair tool (**Click to set the pivot**) and click or drag on the frame — or use the 9 presets in the **Pivot** card.
2. **Apply to the whole animation** (on by default) sets it on every frame; turn it off to change only the current frame.

> - Pivots are stored normalised 0–1 in the **untrimmed** sprite and exported to every target (JSON, Godot, Unity, Spine).

Hitboxes, hurtboxes and custom boxes

1. In the **Boxes · frame N** card, choose the **New box type**: **Hitbox** (red), **Hurtbox** (green) or **Custom** (amber, with a name).
2. Pick the square tool (**Drag to draw a box**) and drag on the frame.
3. Use the arrow tool (**Select a box**) to select a box; name or delete it in the list.
4. The copy button (**Copy boxes to the next frame**) copies this frame's boxes to the next frame and moves there — handy for attack sequences.

- Boxes are exported in `meta.coral.hitboxes` of the JSON exports (JSON Hash / Array): **untrimmed** sprite pixels, top-left origin. `docs/FORMATS.md` shows how to read them in Godot and Unity.

5 Flipbook — SubUV sheets for VFX

Lay out a numbered frame sequence as an even N×M sheet (SubUV) for Niagara / Unity particles, with an optional JSON.

How to use

1. Drop a folder of numbered frames (`explosion_0001.png … explosion_0064.png`) or use **Add folder…** in the **Frames** card. Frames are sorted naturally (2 before 10).
2. Choose the frames: **From** / **To** (1-based; 0 = first / last) and **Step** (take every Nth frame).
3. The **Grid** card: **Columns** / **Rows** (0 = Coral suggests the grid closest to square), **Cell** (longer side, px; 0 = the frames' size; resized with Lanczos3) and **Max size** (1024–8192).
4. Check the sheet and the player (cell by cell at the **FPS**). Below it are the engine settings: Niagara SubUV **Sub Image Size** = cols × rows; Unity Texture Sheet Animation **Tiles X / Y**.
5. Check the **Output image** and click **Write flipbook**.

Output

- The **Output image** defaults to `T_<Folder>_SubUV.png` next to the frame folder (e.g. `T_Explosion_SubUV.png`). Use a `.tga` extension to write TGA.
- **Also write a .json (cols, rows, frames, fps, cell)**: also writes `<name>.json` next to the image.
- **Write flipbook** writes at once, without a confirmation dialog, and replaces files of the same name.

Notes

- No trim, no padding: every cell is the same size. Frames of other sizes are resized to the first frame's size (with a warning).
- If the sheet would exceed **Max size**, cells shrink to fit. A grid too small for the frames leaves the last ones out (with a warning).
- Cells fill left to right, top to bottom. Options are remembered between sessions (except the frame list).

6 Pixel — upscale, palettes, swaps, bleed

Four batch tools for pixel art: Upscale (nearest neighbour), Remap to a palette (OKLab), Swap to make colour variants, and Bleed for transparent pixels.

Common workflow

1. Pick the tool in the **Tool** card: **Upscale**, **Remap**, **Swap** or **Bleed**.
2. Add inputs: drop images / folders (folders include sub-folders) or use **Add folder…** / **Add file…**. **Before / after** shows the first input and its result.
3. Choose the **Output**: **Folder** (a folder; empty = next to each input) or **In place (.bak)** (overwrite the inputs after copying each to `<file>.bak`).
4. Click **Dry run** to see the file list, then the main button (e.g. **Upscale ×4**) to write. The main button writes at once, without asking.

Upscale

Nearest-neighbour upscale, **Factor** ×2, ×3, ×4, ×5, ×6 or ×8 — pixels stay crisp. Output: `<name>_x4.png` (per factor, same PNG / TGA format as the input).

Remap (to a palette)

Every colour moves to the nearest palette colour (measured in OKLab, alpha kept). Output: `<name>_remap.png`. The result lists the pixels changed and which colours were merged into which. **Remap to palette** needs a palette with at least one colour.

- **Extract from inputs**: builds the palette from the inputs, sorted **By use** (most used first) or **By hue**. Up to 4096 colours.
- The upload button (**Load palette**) opens a Lospec `.hex`, GIMP `.gpl` or JASC `.pal`; the download button (**Save palette**) saves the palette.
- **+** adds a colour; click a swatch to remove it from the palette.

Swap (colour variants)

Each swap table is one variant: a list of `from → to` colour pairs. Each table makes `<name>_<variant>.png` and **never touches the source**: variants go to the **Output folder**, or next to the source when it is empty.

1. Get a palette (**Extract from inputs** or load one) — its swatches appear in every table.
2. Name the variant (the box on the left, e.g. `alt`), click a swatch to add a pair, then pick the target colour. **+ pair** adds an empty pair; **×** removes one.
3. **+** on the **Swap tables** card adds a variant; the bin removes one. Click **Make variants** to write.

- **Save to project** stores the tables in the open project (then Ctrl+S); **From project** loads them back.

Bleed

Transparent pixels take their neighbours' colour while alpha stays 0 — no dark fringes when the engine filters textures or builds mipmaps. The image looks the same.

- When writing to a folder (including an empty one = next to each input), outputs get a `_bleed` suffix (e.g. `hero_bleed.png`) and the inputs are left alone. To change the inputs themselves, choose **In place (.bak)**: Coral makes a `.bak` copy first.

7 Normal 2D — normal maps for 2D lighting

Make normal maps from luminance or an alpha bevel, OpenGL / DirectX convention, with a lit preview that follows the pointer; reuse the settings for Pack's normal atlas.

How to use

1. Drop sprites (or folders) onto the window, or use **Add folder…** / **Add file…**. The preview shows the first one lit by a light that follows the pointer, next to its normal map; with several images, the file-name buttons below switch between them.
2. Choose **Height from**: **Luminance** (bright = raised) or **Alpha bevel** (raised from the sprite's edge inward over **Bevel** px — good for flat-coloured sprites).
3. Adjust **Strength** (steeper slopes), **Blur** (softer, px) and **Invert (dark = raised)** if needed.
4. Choose the **Green channel**: **OpenGL (Godot, Unity)** or **DirectX (Unreal)**.
5. In the **Write** card, set the **Output folder** (empty = next to each input), click **Dry run** to see the list, then **Write normal maps**.

Output

- Each input gets `<name>_n.png`. Transparent pixels are flat `(128, 128, 255)`, alpha kept.
- **Write normal maps** writes at once, without asking; existing `_n.png` files are replaced. Inputs are never overwritten (there is no In place mode).

Normal atlas for Pack

Use for the project's normal atlas copies the current settings into the project and turns on **Normal-map atlas** in Pack (the button shows `(on)` once it is on): `hero_n.png` gets the same rects as `hero.png`, so one atlas description serves both.

- Move the pointer over the preview to move the light and check what is raised or sunken before writing.